

训练自由掩码扩散解码的截止期限路径

方法名称: Deadline Path Projection (DPP)

研究动机

掩码扩散语言模型 (DLM) 通过反复把多个掩码位置替换成 token, 来并行生成文本。在严格限制为 B 次模型调用时, 每次调用可能承担三种相互竞争的任务: 揭示更多位置、撤销较弱的早期提交, 或者为下一步获得更完整的上下文。现有解码器通常分别控制这些任务, 因此一次看似合理的修复, 可能恰好占用原本用于完成剩余位置的调用。

Saber 能加速揭示并回滚低置信度选择, 但其揭示阈值和回滚配额并不由剩余调用次数决定。MEDAL 只在解码初始化阶段使用搜索, 因为覆盖整条轨迹的搜索成本过高。ADAS 会更谨慎地选择当前掩码位置, 却把停止和重掩码规则留给基础采样器。近期开放的 LLaDA-8B 与 Dream-7B 能提供逐位置概率, 使新候选和旧提交可以在同一次决策中比较。

Deadline Path Projection (DPP) 先规定每次调用后必须提交多少位置, 再从掩码位置与已提交位置中共同选出相应数量。只有当另一个掩码候选补位时, 较弱的旧 token 才会重新变为掩码, 因此修复不会拖慢必须完成的净推进, 并能在恰好 B 次调用后达到零掩码。真正需要验证的是这些交换是否提升最终 Pass@1; 固定全部旧提交、令 $r_b = 0$, 同时保持相同完成路径, 就能检验这一机制。

方法

背景

1. 为每次运行建立不可变记录, 包含模型检查点、分词器、目标长度 L 、模型长度上限 L_{max} 、允许的模型调用次数 B , 以及可选的时间目标 T 。若直接输入 B , 要求它是正整数。若输入 T , 则固定模型、软件环境、批量大小 1 和长度 L ; 先执行 5 次不计预热, 再在 32 个留出输入题目上, 对每次模型前向计算加排序的完整操作在设备同步前后计时, 取最大测量值作为 τ_{max} , 并按既定向下取整规则得到 B 。检查 $B \geq 1$ 且 $1 \leq L \leq L_{max}$ 。解码前必须固定每道题如何获得 L , 以及提前出现序列结束 (End of Sequence, EOS) 词元时如何处理。【作者需决定: 选择不查看参考答案的 L 规则, 并决定 EOS 后的词元是保留、改为填充, 还是仅在还原文本时忽略】建立长度为 L 的 $token_{ids}$ 数组并全部写入 $mask_token_id$, 建立同长度且全为 false 的 committed 布尔数组, 令已提交位置集合 U_B 为空, 由两者形成全掩码画布 x_B , 并令 b 等于 B 。

为什么: 后续每个选择都需要明确剩余调用次数, 不可行的延迟目标应在解码开始前被识别。

2. 把当前画布构造造成批量大小为 1 的 $input_{ids}$ 张量: 已提交位置写入现有词元编号, 其余位置写入 $mask_token_id$, 并在注意力掩码中把全部 L 个位置标为有效。将 LLaDA-8B 或 Dream-7B 设为评估模式, 关闭梯度记录, 保持模型参数不变。特殊词元参数或扩散时间参数沿用该检查点的参考推理封装; 若模型必须接收时间或噪声值, 就使用参考采样器针对当前掩码比例给出的值, 不额外把 B 或 b 作为条件。模型只调用一次。输出必须是有限数值组成的三维表, 形状为 $[1, L, V]$, 其中 V 是词表大小; 把每个位置的未归一化分数转换为 float32 词表概率分布 p_i , 并把这唯一一份结果交给下一步, 不能再次调用模型。

为什么: 方法只使用允许的冻结模型调用所产生的证据, 不引入第二个评分器或额外探测。

3. 对预先规定的每组输入题目、模型检查点、长度规则和 B , 分别运行截止路径投影 (Deadline Path Projection, DPP)、固定旧提交的禁止交换版本, 以及 Saber、注意力折扣自适应采样器 (Attention-Discounted Adaptive Sampler, ADAS)、Fast-dLLM 和 Top-k 的官方实现。所有方法使用相同的分词器、模型参数、初始画布, 并且最多调用模型 B 次。禁止交换版本保留此前全部已提交位置, 只从掩码位置中按相同的同分规则加入支持度最高的候选, 直到达到下一目标提交数; 这样 r_b 恒为零, 同时完成路径不变。主要评测采用 LLaDA-8B、HumanEval, 以及 B 为 16、32、64 的三种设置。除共同的 B 与 L 规则外, 基线沿用论文发布的默认参数; 达到 B 次调用后立即停止, 若仍有掩码就记为未完成失败, 不能额外调用模型。每次运行保存方法名、检查点、题目编号、 B 、 L 、每一步的 r_b 、最终掩码数、解码词元、答案是否正确, 以及从初始化画布到还原文本的设备同步计时。HumanEval 的 Pass@1 使用其官方执行评测工具计算; 官方指标为精确匹配率的任务则计算精确匹配率。按方法和 B 汇总完成率、质量与延迟, 并对同一批题目进行 10,000 次配对自助重采样, 给出置信区间。

为什么: 相同调用上限检验截止期限主张, 令 $r_b = 0$ 则能区分质量收益来自交换, 还是仅来自固定完成路径。

M1: 截止期限路径

把剩余调用次数转换为精确的已提交集合大小, 并维持终点零掩码不变量。

4. 每次模型调用开始时, 先检查 b 是 1 到 B 之间的整数。用精确整数运算执行已给出的向上取整规则, 得到本次调用后必须剩余的掩码数, 避免浮点舍入改变位置数量。再计算互补的已提交位置数 K_{b-1} , 检查两个数量都在 0 到 L 之间, 并确认已提交位置数不会减少。把 b 、 $b-1$ 、目标掩码数 m_{b-1} 和目标提交数 K_{b-1} 写入同一条状态转移记录。后续操作都读取这条记录, 不能再置置信度阈值改变目标数量。截止期限路径规定当前前向计算结束后的

掩码数与已提交位置数。

$$m_{b-1} = \left\lceil \frac{L(b-1)}{B} \right\rceil, \quad K_{b-1} = L - m_{b-1} \quad (1)$$

为什么：这条路径预留了足够的净推进量，即使替换部分旧提交也能达到 $m_0 = 0$ 。

- 只有第 8 步（先建立一个长度为 L ）的全部检查通过后，才用 x_{b-1} 和 U_{b-1} 替换当前画布与已提交集合，然后把 b 准确减一。若新的 b 仍大于零，就回到第 4 步（每次模型调用开始时）；若 b 等于零，则检查 U_0 包含全部 L 个位置、 $token_{ids}$ 中没有 $mask_token_id$ ，并确认轨迹中恰有 B 次模型调用。使用同一个分词器还原文本，并执行第 1 步（为每次运行建立不可变记录）已固定的长度与 EOS 处理规则。返回完整词元序列、还原后的文本和状态转移轨迹。任何不变量失败都立即报错，不能静默补造或丢弃位置。维持路径不变量即可在截止期限到达时得到完全提交的画布。

$$|\{i : x_{b-1,i} = [MASK]\}| = m_{b-1}, \quad m_0 = 0 \quad (2)$$

为什么：每次转移都维持掩码数不变量，就能保证恰好 B 次前向计算后达到零掩码。

M2: 跨状态投影

共同评估新候选与旧提交，在规定集合大小内完成无独立配额的提交-修复交换。

- 把同一次模型调用得到的概率表转换成逐位置记录。对掩码位置，选择概率最高的词元 v_i ；若多个词元同分，则选择编号较小者，并把其对数概率记为支持度 s_i 。对已提交位置，保留当前词元 z_i ，并从同一份模型输出中读取该词元的对数概率。每条记录包含位置索引、此前是掩码还是已提交、候选词元编号和支持度四个字段。概率为零时允许支持度为负无穷，其他非有限数值都视为错误。按位置索引保存记录，使下一步获得确定性的输入。同一次前向计算为每个位置的掩码候选或当前已提交 $token$ 提供可比较的支持度。

$$s_i = \begin{cases} \log p_i(v_i), & i \notin U_b, v_i = \arg \max_v p_i(v), \\ \log p_i(z_i), & i \in U_b. \end{cases} \quad (3)$$

为什么：把新候选和旧提交放到同一支持度尺度上，推进与修复就能直接竞争。

- 把全部 L 条位置记录只排序一次：先按支持度 s_i 从高到低排列，同分时再按位置索引从小到大排列。取前 K_{b-1} 条记录； $K_{b-1} = 0$ 时得到空集合， $K_{b-1} = L$ 时选中全部位置。把入选位置集合记为 U_{b-1} ，同时转换成长度为 L 的 `selected` 布尔数组，并检查其中恰有 K_{b-1} 个不同位置。之后不能再按新旧状态设置不同配额、增加置信度截止条件或进行第二轮排序。下一已提交集合是在两种状态之间进行的固定基数 $top-K$ 选择。

$$U_{b-1} = \text{TopK}_{i \in \{1, \dots, L\}}(s_i; K_{b-1}) \quad (4)$$

为什么：截止期限只规定必须保留多少提交，当前模型证据决定具体保留哪些位置。

- 先建立一个长度为 L 、全部填入 $mask_token_id$ 的新数组，以原子方式构造下一画布。对 U_{b-1} 中的每个位置，若它此前已经提交，就从 x_b 复制当前词元 z_i ；若它此前是掩码，就写入第 6 步（把同一次模型调用得到的概率表 ...）保存的新候选 v_i 。 U_{b-1} 之外的位置继续保持掩码，因此未入选的旧提交会被明确撤销。把旧提交中未入选 U_{b-1} 的位置数量定义为交换数 r_b ，并同时保存撤销位置、新提交位置、保留位置三个列表以及提交数的净增量。替换旧状态前，检查每个入选位置都不是掩码、每个未入选位置都是 $mask_token_id$ 、`committed` 布尔数组与 U_{b-1} 完全一致、已提交位置恰有 K_{b-1} 个，并且掩码恰有 m_{b-1} 个。最后一次性发布新画布、已提交集合、交换数和转移记录。除必须完成的净推进外，每个被撤销的提交都对应一个额外进入集合的掩码候选。

$$r_b = |U_b \setminus U_{b-1}|, \quad |U_{b-1} \setminus U_b| = r_b + (K_{b-1} - K_b) \quad (5)$$

为什么：每个被修复的旧 $token$ 都在同一个固定大小集合中获得补位，因此修复不消耗额外进度，而且可以直接测量。